

QUESTIONS

```
1.#include<stdio.h>
int main()
{
union a
{
int i;
char ch[2];
};
union a u;
u.ch[0]=6;
u.ch[1]=2;
printf("%d\n", u.i);
return 0;
}
```

```
2.#include<stdio.h>
typedef int x;
```

```
x main() {
    x a = 10;
    printf("%d", a);
}
```

```
3.#include<stdio.h>
#define x int
```

```
x main() {
    x a = 10;
    printf("%d", a);
}
```

```
4.#include <stdlib.h>
#include <stdio.h>
```

```
int main() {
    int *p = malloc(sizeof(int));
    *p = 10;
    free(p);
    printf("%d", *p);
}
```

What is the output?

- A) Prints 10
- B) Prints garbage
- C) Compilation error
- D) Undefined behavior

```
5.#include<stdio.h>
void fun();
int main()
{
```

```

        typedef char new;
        new x=20;
        fun();
    }
    void fun()
    {
        new y=200;
        printf("%d\n",y);
    }

```

```

6.#include<stdio.h>
int main()
{
    union var
    {
        int a, b;
    };
    union var v;
    v.a=10;
    v.b=50;
    printf("%d\n", v.a);
    return 0;
}

```

7.What is the key difference between structure and union?

- A) Structure stores only one member at a time
- B) Union stores all members separately
- C) Structure members have separate memory locations
- D) Union members cannot be accessed

8.What does malloc() return if memory allocation fails?

- A) 0
- B) -1
- C) NULL
- D) Garbage value

```

9.#include <stdio.h>
#include <stdlib.h>
int main() {
    int *p = realloc(NULL, 10 * sizeof(int));
    printf("%p", p);
}

```

What is true?

- A) Always NULL
- B) Same as malloc()
- C) Compilation error
- D) Runtime error

```

10.#include<stdio.h>
int main()
{
struct value
{
int bit1:1;
int bit3:4;
int bit4:4;
}bit;
printf("%d\n", sizeof(bit));
return 0;
}

```

11.Which function initializes allocated memory to zero?

- A) malloc()
- B) calloc()
- C) realloc()
- D) free()

```

12.#include <stdio.h>
int main()
{
union test
{
int i;
int j;
};
union test var=10;
printf("%d,%d\n",var.i,var.j);
}

```

```

13.#include<stdio.h>
int main()
{
struct value
{
int bit1:1;
int bit3:5;
int bit4:4;
}bit={ 1, 2, 13};
printf("%d, %d, %d\n", bit.bit1, bit.bit3, bit.bit4);
}

```

```

14.#include <stdio.h>
#include <stdlib.h>
int main() {
int *p = calloc(3, sizeof(int));
p[0] = 1;
p[1] = 2;
p = realloc(p, 2 * sizeof(int));
printf("%d %d", p[0], p[1]);
}

```

```
}
```

What is the Output?

A) 0 0

B) 1 2

C) Garbage Garbage

D) Runtime error

15.#includue<stdio.h>

```
int main()
```

```
{
```

```
enum days {MON=-1, TUE, WED=6, THU, FRI, SAT};
```

```
printf("%d, %d, %d, %d, %d, %d\n", MON, TUE, WED, THU, FRI, SAT);
```

```
return 0;
```

```
}
```

16.#include<stdio.h>

```
typedef char* old;
```

```
char* fun(char);
```

```
int main()
```

```
{
```

```
    old p=fun('a');
```

```
    printf("%s\n",p);
```

```
}
```

```
char* fun(char z)
```

```
{
```

```
    old s="hello";
```

```
    return s;
```

```
}
```

17.#include<stdio.h>

```
int main()
```

```
{
```

```
struct emp
```

```
{
```

```
char *n;
```

```
int age;
```

```
};
```

```
struct emp e1 = {"name", 23};
```

```
struct emp e2 = e1;
```

```
printf("%s\n", e2.n);
```

```
return 0;
```

```
}
```

18.#include<stdio.h>

```
int main()
```

```
{
```

```
struct node
```

```
{
```

```
int data;
```

```
struct node *link;
```

```
};
```

```
struct node *p;
```

```

p = (struct node *) malloc(sizeof(struct node));
printf("%d, %d\n", sizeof(p), sizeof(q));
return 0;
}

```

19.What is the correct way to avoid a dangling pointer after free()?

- A) Assign 0
- B) Assign -1
- C) Assign NULL
- D) Reallocate memory

```

20.#include<stdio.h>
int main()
{
union a
{
int i;
char ch[2];
};
union a u1 = {512};
union a u2 = {0, 2};
if(u1.ch[0]==u2.ch[1])
printf("true");
else
printf("false");
}

```

21.struct ex

```

{
char ch;
int i;
long int a;
};

```

How to reduce structure size?

- A) Use pointers
- B) Reorder structure members
- C) Use typedef
- D) none

```

22.union uTemp
{double a;
int b[10];
char c;
} u;
int main() {
    u.b[0] = 65;
    printf("%c", u.c);
    return 0;
}

```

```

23.#include <stdio.h>
struct temp
{
int a;
int b;
int c;
};
main()
{
struct temp p[] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
printf("%d %d",p[0].b,p[2].c);
}

```

```

24.#include<stdio.h>
struct temp
{
int a;
int b;
int c;} p[] = {0};
int main()
{
printf("%d", sizeof(p));
}

```

```

25.#include<stdio.h>
struct student
{
};
void main()
{
struct student s[2];
printf("%d", sizeof(s));
}

```

```

26.#include<stdio.h>
struct
{
int k;
char c;
};
int main()
{
struct p;
p.k = 10;
printf("%d\n", p.k);
}

```

```

27.#include <stdio.h>
int main()
{
typedef struct tag{
char str[10];

```

```

int a;
}har;
har h1,h2={"IHelp",10};
h1=h2;
h1.str[1]='h';
printf("%s,%d",h1.str,h1.a);
return 0;
}

```

28.include <stdio.h>

```

int main()
{
union values
{
unsigned char a;
unsigned char b;
unsigned int c;
};
union values val;
val.a=1;
val.b=2;
val.c=300;
printf("%d,%d,%d",val.a,val.b,val.c);
return 0;
}

```

29.#include <stdio.h>

```

enum Status { OK, ERROR, UNKNOWN };

```

```

int main() {
    enum Status s = ERROR;
    printf("%d", s);
    return 0;
}

```

30.#include <stdio.h>

```

int main()
{
union values
{
int intVal;
char chrVal[2];
};
union values val;
val.chrVal[0]='A'; val.chrVal[1]='B';
printf("\n%c,%c,%d",val.chrVal[0],val.chrVal[1],val.intVal);
return 0;
}

```

31.#include<stdio.h>

```

typedef int char;
int main()

```

```

{
    char x=12;
    printf("%x\n",x);
}

```

```

32.#include <stdio.h>
struct sample
{
int a=0;
char b='A';
float c=10.5;
};
int main()
{
struct sample s;
printf("%d,%c,%f",s.a,s.b,s.c);
return 0;
}

```

```

33.#include<stdio.h>
typedef struct p
{
int x, y;
}k;
int main()
{
struct p p = {1, 2};
k k1 = p;
printf("%d\n", k1.x);
}

```

```

34.#include<stdio.h>
typedef pf printf;
int main()
{
pf("hello india\n");
}

```

```

35.#include<stdio.h>
enum A { P = 3, Q = P + 2, R };

int main() {
    printf("%d %d %d", P, Q, R);
}

```

```

36.#include<stdio.h>
typedef int var;
int main()
{

typedef char var;
var x=228;

```



```
printf("%d\n",x);  
}
```

```
37.#include<stdio.h>  
union arc  
{  
char x;  
int y;  
char ax[8];  
}aha;  
what is the size of this union.
```

```
38.#include <stdio.h>  
#define A 10  
#define B A+5
```

```
int main() {  
    printf("%d", B*2);  
}
```

```
39.#include <stdio.h>  
int main()  
{  
struct s1 { int i ;};  
struct s2 { int i ;};  
struct s1 st1;  
struct s2 st2;  
st1.i = 5;  
st2 = st1;  
printf(" %d", st2.i);  
}
```

```
40.#include<stdio.h>  
enum X { A = 1, B = 2 };  
enum Y { C = 1, D = 2 };
```

```
int main() {  
    printf("%d", A == C);  
}
```

```
41.#include<stdio.h>  
struct x  
{  
int a;  
long b;  
};  
union y  
{  
int a;  
long b;  
struct x v;  
};
```

```
int main()
{
    union y u={100};
    printf("%d",u.v.a);
}
```

```
42.#include<stdio.h>
union y
{
    int a;
    long b;
    struct x v;
};
struct x
{
    int a;
    long b;
};
int main()
{
    union y u={100};
    printf("%d",u.v.a);
}
```

43.Which of the following causes a memory leak?

- A) Forgetting to free allocated memory
- B) Calling free(NULL)
- C) Assigning pointer to NULL after free
- D) Using calloc()

```
44.#include <stdio.h>
struct p
{
    int k;
    char c;
    float f;
};
int main()
{
    struct p x = {.c = 97};
    printf("%f\n", x.f);
}
```

```
45.#include <stdio.h>
int main()
{
    struct std
    {
        char name[30];
        int age;
    };
}
```

```

struct std s1={"Mike",26};
struct std *p=&s1;
printf(" %c, %d\n",p->name[2],(*p).age);
}

```

```

46.struct TAB{
short a;
int b;
long c;
}TABLE[10];
What is the size of TABLE?

```

```

47.#include <stdio.h>
#define MUL(a,b) (a*b)
int main() {
int a=2,b=3,c=4,d=1;
    printf("%d", MUL(a+b, c+d));
}

```

```

48.#include <stdio.h>
union test
{
int x;
char arr[4];
int y;
};
int main()
{
union test t;
t.x = 0;
t.arr[1] = 'G';
printf("%s", t.arr);
return 0;
}

```

```

49.#include <stdio.h>
#include <stdlib.h>
struct Student {
    int id;
    float marks;
};
int main() {
    int n = 3;
    struct Student *s;
    s = (struct Student *)malloc(n * sizeof(struct Student));

    for (int i = 0; i < n; i++) {
        s[i].id = i + 1;
        s[i].marks = 80 + i;
    }
}

```

```
printf("%d %f",s[1].id,s[2].marks);  
free(s);  
}
```

50.Which statement about realloc() is TRUE?

- A) It always returns a new memory location
- B) It preserves old data
- C) It initializes new memory to zero
- D) It cannot reduce memory size